

Dokumentation zum Onlineshop „Blubber Baron“

Ein Projekt von Studierenden im vierten Semester im
Studiengang WK Modul Internettechnologien SS2026

Gruppe:

„Name wird noch nachgereicht“

Projektverantwortliche:

Leen Anis (514907)

Esma Cil (518059)

Patrick Wölflle (491084)

Dominik Vogler (507398)

Manuel Schneid (523945)

Betreuender Dozent:

Prof. Dr. Matthias Becker

Kempten, 09.07.2026

Inhaltsverzeichnis

1. Abbildungs- und Tabellenverzeichnis	2
2. Einführung	3
2.1. Einleitung und Problembeschreibung	3
2.2. Allgemeine Beschreibung des Projekts.....	5
3. Anforderungsanalyse	7
4. Tech-Stack	8
5. Beschreibung der Implementierung	11
5.1. Frontend	11
5.2. Backend	13
5.3. Datenstruktur	15
6. API-Beschreibung	16
7. Diskussion und Ausblick	17
8. Erklärung der selbständigen Anfertigung	19
9. Anhang	20
9.1. RACI	20
9.2. Projektplan	21
9.3. Strukturplan	22

1. Abbildungs- und Tabellenverzeichnis

Abbildung 1: Tabelle für Kann-Muss-Kriterien	7
Abbildung 2: Tabelle für verwendete Technologien im Überblick	10
Abbildung 3: Frontend.....	11
Abbildung 4: Backend	13
Abbildung 5: Datenstruktur	15
Abbildung 6: API-Beschreibung.....	16

2. Einführung

2.1. Einleitung und Problembeschreibung

Im Rahmen der Projektarbeit wurde die Webanwendung Blubber Baron entwickelt. Dabei handelt es sich um einen Online-Shop für Shisha-Produkte und passendes Zubehör. Ziel des Projekts war es, eine moderne, übersichtliche und präsentierbare Shop-Webseite zu erstellen, die sowohl optisch zur Marke passt als auch grundlegende Funktionen eines Online-Shops abbildet.

Die Idee hinter Blubber Baron war es, einen hochwertigen digitalen Shop mit dunklem, edlem Design und klarer Markenwirkung umzusetzen. Besonders bei einem Online-Shop ist es wichtig, dass Nutzer Produkte schnell finden, Produktinformationen verständlich dargestellt werden und die Bedienung möglichst einfach bleibt. Eine unübersichtliche oder technisch fehlerhafte Webseite kann dazu führen, dass Kunden den Shop schnell wieder verlassen oder Bestellungen nicht abschließen.

Die Problemstellung bestand daher darin, eine Webanwendung zu entwickeln, die Produktdaten strukturiert darstellt, eine Verbindung zwischen Frontend und Backend ermöglicht und grundlegende Shop-Funktionen zuverlässig bereitstellt. Dazu gehörten unter anderem die Produktübersicht, die Anzeige einzelner Produktdetails, ein Warenkorb sowie Verwaltungsfunktionen für Produkte. Zusätzlich sollte das Projekt lokal lauffähig sein und mithilfe einer Docker-Umgebung einfacher gestartet und präsentiert werden können.

Ein weiterer wichtiger Bestandteil war die technische Umsetzung über eine REST-API. Das Frontend sollte Produktdaten nicht nur statisch anzeigen, sondern diese über die Backend-Schnittstelle laden und weiterverarbeiten. Dadurch wurde Blubber Baron nicht nur als reine Design-Webseite umgesetzt, sondern als Grundlage für einen funktionsfähigen Online-Shop mit erweiterbarer technischer Struktur.

Neben der Kundensicht spielte auch die Verwaltung des Shops eine Rolle. Über Admin-Funktionen sollten Artikel verwalten, gelöscht und zukünftig auch Preise bearbeitet werden können. Dadurch entstand eine Webanwendung, die sowohl die Präsentation der Produkte als auch erste Verwaltungsprozesse berücksichtigt.

Zusammenfassend verfolgt Blubber Baron das Ziel, einen modernen Online-Shop für Shisha-Produkte zu schaffen, der technisch sauber aufgebaut, lokal ausführbar und für eine Live-Demo geeignet ist. Die Anwendung bildet damit eine solide Grundlage, die zukünftig zu einem vollständigen Shopsystem erweitert werden kann.

2.2. Allgemeine Beschreibung des Projekts

Das Projekt „Blubber Baron“ ist ein prototypischer Online-Shop für verschiedene Shisha-Produkte. Die Webanwendung richtet sich an Nutzer, die sich über Shishas, Kohle, Tabak und weiteres Zubehör informieren möchten. Hierfür wird ein digitaler Shop-Service bereitgestellt, der einen typischen E-Commerce-Anwendungsfall abbildet.

Der Fokus der Anwendung liegt auf einer übersichtlichen Produktpräsentation, einer intuitiven Navigation sowie einer klaren Strukturierung des Produktsortiments. Ziel ist es, den Nutzenden eine einfache und benutzerfreundliche Möglichkeit zur Informationsbeschaffung und Produktauswahl zu bieten.

Die Webapplikation besteht aus den zwei folgenden Hauptkomponenten:

- Frontend: Stellt die Benutzeroberfläche dar und ermöglicht die Interaktion mit den Nutzenden
- Backend: Übernimmt die Geschäftslogik, verarbeitet Anfragen und stellt die Kommunikation mit der Datenbank sicher

Über das Frontend können Nutzer Produktinformationen einsehen, gezielt nach Produkten suchen, Rezensionen verfassen sowie sich über Karrieremöglichkeiten informieren. Das Backend verarbeitet die eingehenden Anfragen, stellt die erforderlichen Daten über eine REST-API bereit und kommuniziert mit der Datenbank. Diese dient der dauerhaften Speicherung von Produktinformationen wie Preisen, Produktbeschreibungen, Produktbildern sowie weiteren relevanten Daten.

Ein besonderer Schwerpunkt liegt auf der praktischen Anwendung der im Modul Internettechnologien vermittelten Inhalten. Dazu gehören die Gestaltung der Oberfläche mit HTML und CSS, die Umsetzung interaktiver Funktionen durch TypeScript sowie die Anbindung einer Datenbank über REST-Schnittstellen. Dadurch wird das Projekt nicht nur als einfache statische Webseite umgesetzt, sondern als vollständige Webanwendung mit mehreren technischen Schichten und einer klaren Trennung zwischen Präsentations-, Geschäftslogik- und Datenhaltungsebene realisiert.

Neben den behandelten Technologien in den Vorlesungen, konnten ebenso weitere

hilfreiche und geeignete Technologien verwendet werden. Somit konnte frei gewählt werden, mit welchen Technologien das Projekt und deren Komponenten (Frontend, Backend und Datenbank) erstellt wird.

Der entwickelte Online-Shop ist als Prototyp zu verstehen. Das bedeutet, dass die wichtigsten Grundfunktionen eines E-Commerce-Systems umgesetzt werden, ohne zwingend alle Anforderungen eines realen Produktivsystems vollständig abzubilden. Dazu zählen zum Beispiel der Check-out-Prozess oder die Altersverifikation. Für die Projektarbeit im Modul Internettechnologien ist vor allem entscheidend, dass die einzelnen technischen Bestandteile nachvollziehbar umgesetzt, dokumentiert und von den Studierenden verstanden und verinnerlicht werden.

3. Anforderungsanalyse

Für die Umsetzung des Projekts wurden verschiedene Anforderungen an das Projekt bereits in den Gruppenterminen mit dem Dozenten definiert und festgelegt. Dabei wurde zwischen Muss-Kriterien und Kann-Kriterien unterschieden.

Muss-Kriterien: Es sind Funktionen, die für die grundlegende Erfüllung der Aufgabenstellung notwendig sind.

Kann-Kriterien: Sie erweitern die Anwendung zusätzlich, sind aber für die grundlegende Funktionsfähigkeit nicht zwingend erforderlich.

Im nachfolgenden einen Überblick über die definierten Muss- und Kann-Kriterien:

Muss-Kriterien	Kann-Kriterien
Funktionsfähige Weboberfläche mit Startseite, Produktübersicht, Navigation und Produktdaten	Such- oder Filterfunktion nach Produkten und Kategorien
Umsetzung mit HTML, CSS und TypeScript	Login-System für das Nutzerkonto
Backend mit serverseitiger Datenverarbeitung	Administrationsbereich zur Pflege der Produkte
Verwendung der REST-API-Befehle (GET, POST, PUT, PATCH und DELETE)	Bestellhistorie oder Kundenbereich als spätere Erweiterung
Datenbank zur dauerhaften Speicherung von Produktdaten	Erweiterte Design-Elemente im Corporate Design vom Blubber Baron
Behandlung von Fehleingaben	Erweiterung über einen Social-Media-Auftritt
Responsive für Smartphone, Tablet und Desktop	Bewerbungs-/Karrierebereich

Abbildung 1: Tabelle für Kann-Muss-Kriterien

4. Tech-Stack

Für die Umsetzung des Online-Shops wurden verschiedene Programmiersprachen, Frameworks und Werkzeuge eingesetzt. Die Anwendung folgt einer klassischen Client-Server-Architektur und besteht aus einem Frontend, einem Backend sowie einer Datenbank.

Frontend:

Das Frontend wurde mit HTML, CSS, TypeScript sowie dem React-Framework Next.js entwickelt. HTML dient der strukturellen Gliederung der Inhalte, während CSS für die Gestaltung und das responsive Layout der Benutzeroberfläche im Online-Shop verantwortlich ist. Die Interaktivität der Anwendung wird mithilfe von TypeScript umgesetzt. Next.js bildet dabei die Grundlage für die Entwicklung der Benutzeroberfläche und unterstützt unter anderem die komponentenbasierte Entwicklung.

Über das Frontend können Nutzer Produktinformationen abrufen, nach Produkten suchen, Rezensionen verfassen sowie weitere Inhalte der Website aufrufen. Die Kommunikation mit dem Backend erfolgt über eine REST-API.

Backend:

Das Backend wurde mit Java und dem Framework Spring Boot realisiert. Es übernimmt die Geschäftslogik der Anwendung, verarbeitet eingehende Anfragen des Frontends und stellt die benötigten Daten über definierte REST-Endpunkte bereit.

Die Anwendung ist nach dem Schichtenmodell aufgebaut und umfasst die Komponenten Controller, Service, Repository, Model sowie DTO (Data Transfer Object). Diese Architektur ermöglicht eine klare Trennung der Verantwortlichkeiten und verbessert die Wartbarkeit sowie Erweiterbarkeit der Anwendung.

Die Kommunikation zwischen Frontend und Backend erfolgt über HTTP-Anfragen. Hierbei werden die REST-Methoden GET zum Abrufen von Daten, POST zum Erstellen neuer Datensätze, PUT beziehungsweise PATCH zum Aktualisieren bestehender Daten sowie DELETE zum Entfernen von Datensätzen verwendet.

Datenbank:

Für die dauerhafte Speicherung der Anwendungsdaten wird MySQL eingesetzt. Die Datenbank wird innerhalb eines Docker-Containers betrieben. In der Datenbank werden unter anderem Produktinformationen wie Produktname, Produktbeschreibung und Preis sowie weitere produktspezifische Eigenschaften gespeichert. Der Zugriff auf die Datenbank erfolgt ausschließlich über das Backend. Hierbei werden die Datenbanktabellen durch sogenannte Entity-Klassen abgebildet. Über das Repository werden die gespeicherten Daten gelesen, geändert oder gelöscht und anschließend über die Geschäftslogik des Backends dem Frontend über REST-Endpunkte zur Verfügung gestellt.

Durch die Trennung von Frontend, Backend und Datenbank entsteht eine übersichtliche und modular aufgebaute Anwendung. Änderungen an einer Komponente können weitgehend unabhängig von den übrigen Komponenten vorgenommen werden. Dies verbessert sowohl die Wartbarkeit als auch die Erweiterbarkeit der Software.

Das Frontend stellt die Benutzeroberfläche bereit und sendet Anfragen an das Backend. Das Backend verarbeitet diese Anfragen, greift bei Bedarf auf die Datenbank zu und liefert die entsprechenden Daten über die REST-API an das Frontend zurück. Auf diese Weise entsteht eine vollständige mehrschichtige Webanwendung.

Weitere Tools:

Für die Entwicklung des Projekts wurden verschiedene Werkzeuge eingesetzt. Als Entwicklungsumgebungen kamen Visual Studio Code für das Frontend sowie Eclipse für das Backend zum Einsatz. Docker wurde für die Containerisierung der MySQL-Datenbank verwendet. Die Versionsverwaltung des Quellcodes erfolgte mit Git, wodurch Änderungen nachvollziehbar dokumentiert und die Zusammenarbeit im Team erleichtert wurden.

Die verwendeten Technologien im Überblick:	
Programmiersprachen	Java, TypeScript, HTML, CSS
Frontend	Next.js, React
Backend	Spring Boot
Datenbank	MySQL
API	REST
Containerisierung	Docker
Versionsverwaltung	Git
Entwicklungsumgebungen	Visual Studio Code, Eclipse

Abbildung 2: Tabelle für verwendete Technologien im Überblick

5. Beschreibung der Implementierung

5.1. Frontend

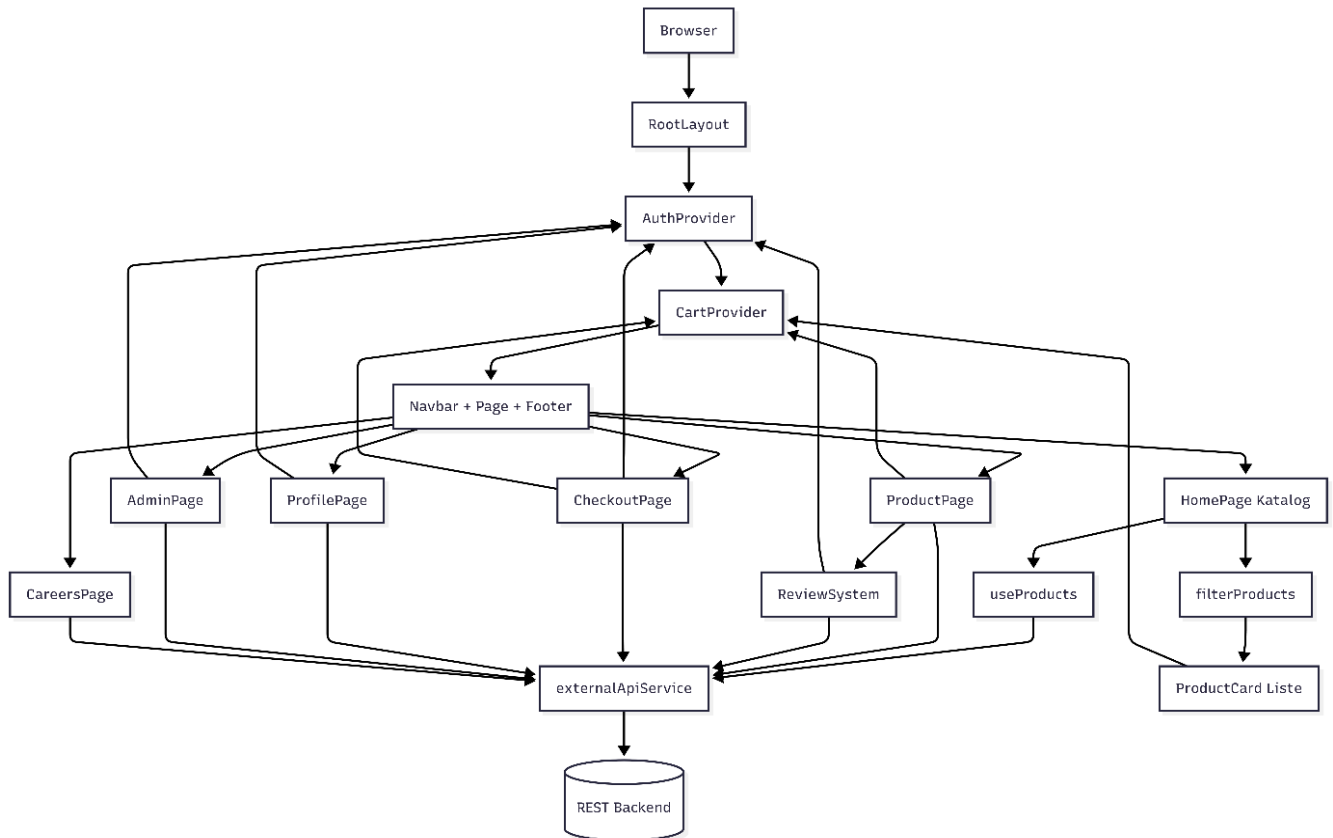


Abbildung 3: Frontend

Kurzfassung Flowchart

- Der Browser lädt die Anwendung über das RootLayout.
- Im RootLayout werden AuthProvider und CartProvider um die gesamte App gelegt.
- Dadurch können alle Seiten und Komponenten auf Login-Daten und Warenkorb-Daten zugreifen.
- Die Navbar ist Teil der gemeinsamen App-Shell und enthält Suche, Profil-Link und Warenkorb-Zugriff.
- Die HomePage lädt Produkte über useProducts aus dem externalApiService.

- externalApiService sendet die REST-Anfragen an das Backend.
- Die geladenen Produkte werden auf der Startseite gefiltert und als ProductCard dargestellt.
- Produktkarten und Detailseiten können Produkte über den CartProvider in den Warenkorb legen.
- Die ProductPage lädt ein einzelnes Produkt, verwandte Produkte und bindet das ReviewSystem ein.
- Das ReviewSystem nutzt den eingeloggteten Nutzer aus dem AuthProvider und speichert Bewertungen über die API.
- Die CheckoutPage kombiniert Warenkorb-Daten und Login-Daten, erstellt daraus eine Bestellung und sendet sie ans Backend.
- Die ProfilePage verwaltet Nutzerprofil, Login/Logout und Bestellhistorie.
- Die AdminPage prüft den Admin-Status und verwaltet Produkte sowie Jobangebote über die API.
- Die CareersPage lädt Jobangebote und sendet Bewerbungen an das Backend.
- Die geladenen Jobs werden als Karten mit Abteilung, Standort, Typ und Beschreibung angezeigt.
- Insgesamt fließen UI-Aktionen meistens von Page/Component zu Context, Hook oder Service und von dort weiter zum REST-Backend.

5.2. Backend

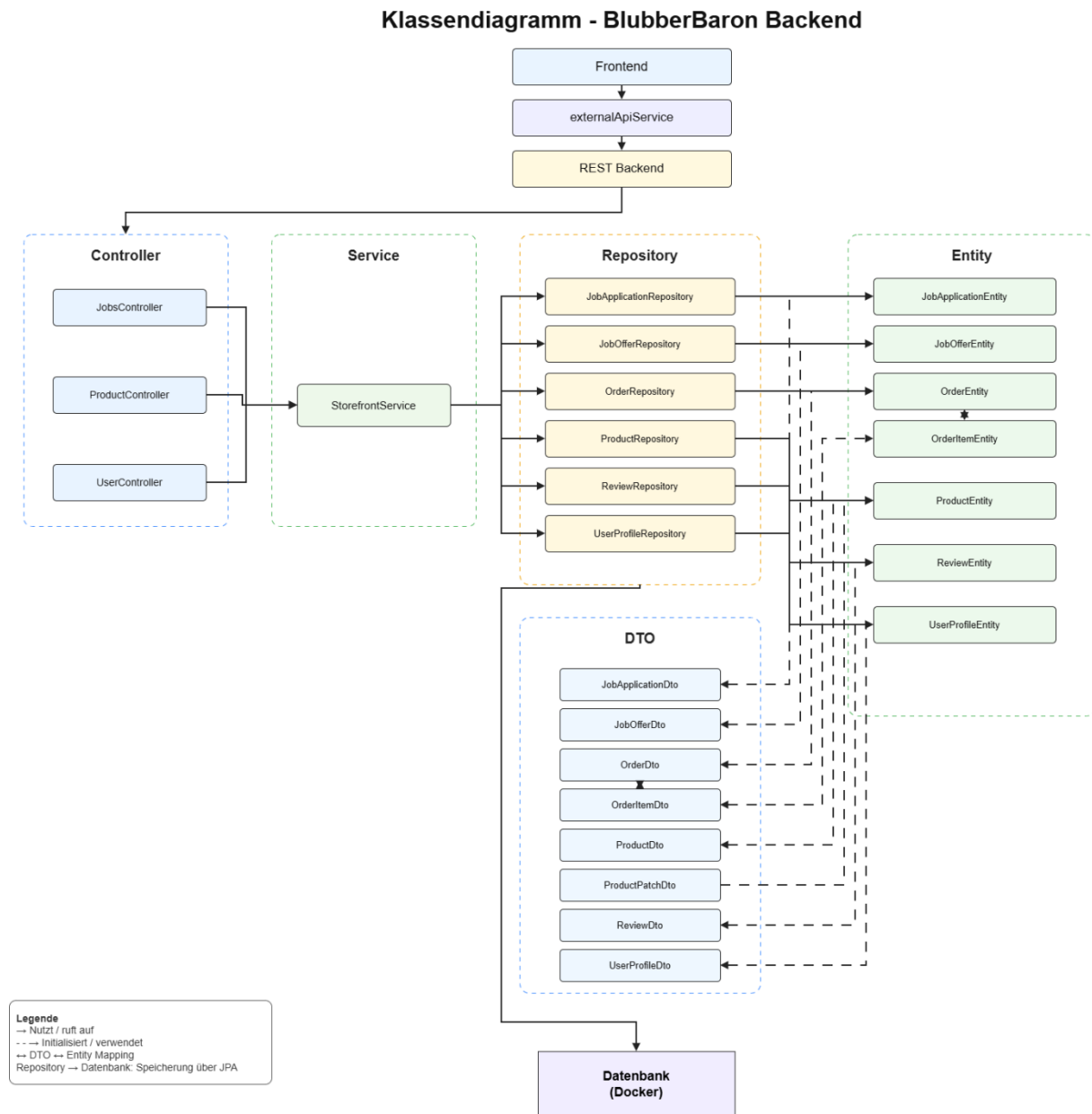


Abbildung 4: Backend

Kurzfassung Flowchart

- Das Frontend sendet Anfragen über den externalApiService an das REST-Backend
- Die Anfrage landet je nach Bereich bei einem Controller, zum Beispiel Product-Controller, UserController oder JobsController

- Die Controller stellen die API-Endpunkte bereit und nehmen Daten vom Frontend entgegen
- Die Controller verarbeiten die Daten nicht direkt, sondern leiten sie an den StorefrontService weiter
- Der StorefrontService ist die zentrale Logik-Klasse des Backends
- Dort wird entschieden, ob Daten geladen, erstellt, geändert oder gelöscht werden
- Für Produkte nutzt der Service das ProductRepository
- Für Bewertungen nutzt der Service das ReviewRepository
- Für Bestellungen nutzt der Service das OrderRepository
- Für Nutzerprofile nutzt der Service das UserProfileRepository
- Für Jobangebote nutzt der Service das JobOfferRepository
- Für Bewerbungen nutzt der Service das JobApplicationRepository
- Die Repositories übernehmen den direkten Zugriff auf die Datenbank
- Die Daten werden als Entities gespeichert, zum Beispiel ProductEntity, OrderEntity oder UserProfileEntity
- DTOs werden genutzt, Daten zwischen Frontend und Backend auszutauschen
- Der StorefrontService wandelt DTOs in Entities um und Entities wieder zurück in DTOs
- Neue Produkte, Bewertungen, Bestellungen, Jobangebote und Bewerbungen werden über die jeweiligen POST-Anfragen gespeichert
- Änderungen, zum Beispiel an Produkten oder Nutzerprofilen, werden im Service verarbeitet und über das Repository gespeichert
- Insgesamt fließen Daten meistens von Controller zu Service, von dort zum Repository und schließlich in die Datenbank

5.3. Datenstruktur

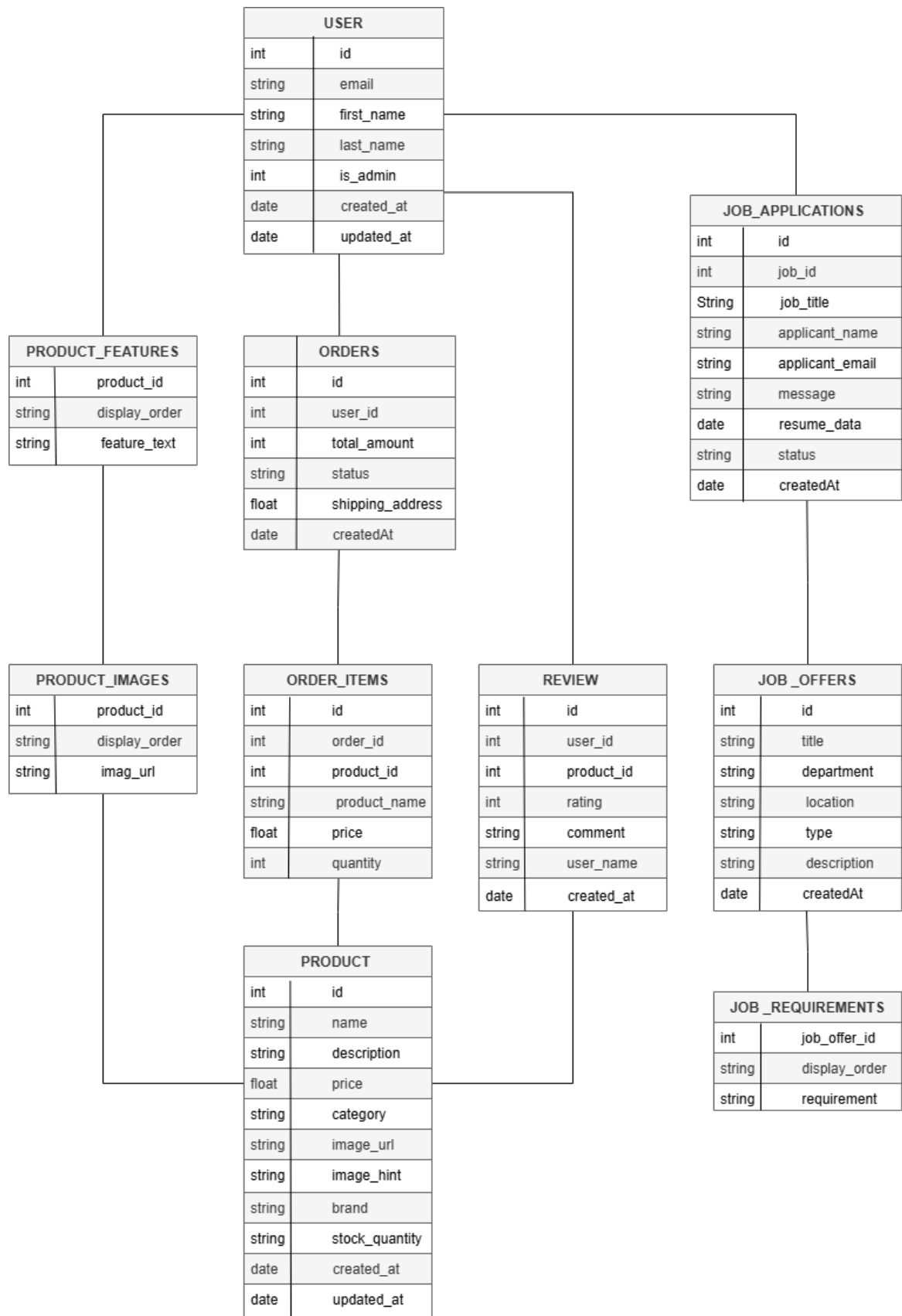


Abbildung 5: Datenstruktur

6. API-Beschreibung

Die API-Schnittstellen können über <http://localhost:9002/api/swagger> abgerufen und getestet werden.

Nachfolgend eine beispielhafte Übersicht über die API-Schnittstellen:

The screenshot displays the Swagger UI for the 'Blubber Baron Backend API'. At the top, the API title is followed by version tags '1.1.0' and 'OAS 3.0'. Below this, a description states: 'Vollständige Swagger/OpenAPI-Dokumentation für den Blubber Baron REST-Backend-Katalog inklusive Produkte, Reviews, Jobs, Bewerbungen, Nutzerprofile und Bestellungen.' A 'Servers' section shows a dropdown menu with the URL 'http://localhost:8081/api - Configured backend API'. The main content is organized into sections for different API resources, each with a list of endpoints and their corresponding HTTP methods:

- Products** (Produktkatalog, Detailsansicht und Admin-Pflege):
 - GET /products: List products (listProducts)
 - POST /products: Create or sync a product (syncProduct)
 - GET /products/{productId}: Get product by ID (getProduct)
 - PATCH /products/{productId}: Partially update a product (patchProduct)
 - DELETE /products/{productId}: Delete product (deleteProduct)
- Careers** (Stellenangebote und Karrieredaten):
 - GET /jobs: List job offers (listJobs)
 - POST /jobs: Create or sync a job offer (syncJob)
 - GET /jobs/{jobId}: Get job offer by ID (getJob)
 - DELETE /jobs/{jobId}: Delete job offer (deleteJob)
- Applications** (Bewerbungseingänge für Stellenangebote):
 - POST /applications: Submit a job application (syncApplication)
- Orders** (Bestellungen und Bestellhistorie):
 - GET /users/{userId}/orders: List orders for a user (listUserOrders)
 - POST /orders: Create an order (syncOrder)
- Users** (Benutzerprofile und Profilpflege):
 - GET /users/{userId}: Get user profile (getUserProfile)
 - PUT /users/{userId}: Update user profile (updateUserProfile)

Abbildung 6: API-Beschreibung

7. Diskussion und Ausblick

Die Umsetzung von Blubber Baron zeigt, dass die grundlegenden Anforderungen an einen Online-Shop erfolgreich realisiert werden konnten. Die Anwendung verfügt über eine Startseite mit starker Markenwirkung, eine Produkt- bzw. Kollektionsübersicht, eine Anbindung an die Backend-API sowie erste Admin-Funktionen zur Verwaltung von Artikeln. Auch die lokale Ausführbarkeit und die Docker-Umgebung tragen dazu bei, dass das Projekt gut demonstriert und technisch nachvollziehbar gestartet werden kann.

Während der Entwicklung wurde deutlich, dass ein Online-Shop aus mehreren eng verbundenen Bereichen besteht. Das Design muss ansprechend und benutzerfreundlich sein, gleichzeitig müssen Frontend, Backend und Datenhaltung zuverlässig zusammenarbeiten. Besonders die REST-API zwischen Frontend und Backend war dabei ein wichtiger Bestandteil, da Produktdaten korrekt geladen, angezeigt und weiterverarbeitet werden mussten.

Eine Herausforderung war der Umgang mit dem LocalStorage, da Warenkorb-Daten im Browser gespeichert und sinnvoll begrenzt werden mussten. Außerdem mussten Produktbilder zuverlässig an den Warenkorb übergeben und dort korrekt angezeigt werden. Auch die Zusammenarbeit mit Git stellte teilweise eine Schwierigkeit dar, da Pushes, Änderungen und mögliche Konflikte klare gemeinsame Regeln erforderten. Zusätzlich zeigte sich, dass Absprachen zu Schnittstellen, Zuständigkeiten und Zwischenständen früher und genauer hätten erfolgen müssen.

Aus diesen Herausforderungen konnten wichtige Erkenntnisse gewonnen werden. Für zukünftige Projekte wäre es sinnvoll, bereits zu Beginn eine klarere Struktur festzulegen. Dazu gehören feste Git-Richtlinien, kleinere Arbeitspakete, regelmäßige kurze Abstimmungen und klare Teil-Deadlines. Auch ein Scrum-Leitfaden oder ein gemeinsames Board hätte dabei geholfen, Aufgaben, Zuständigkeiten und Fortschritte transparenter zu machen.

Für die Weiterentwicklung von Blubber Baron gibt es mehrere sinnvoll Erweiterungsmöglichkeiten. Eine wichtige Funktion wäre ein vollständig funktionierender User-Login. Dadurch könnten Kunden eigene Benutzerkonten erstellen, sich anmelden und persönliche Daten sowie zukünftige Bestellungen verwalten. Dadurch würde der Shop deutlich näher an ein echtes produktives Shopsystem heranrücken.

Ein weiterer zentraler Ausblick ist die Integration einer Bezahl-API bzw. eines echten Bezahl Dienstleisters. Aktuell bildet Blubber Baron vor allem die technische und gestalterische Grundlage eines Online-Shops ab. Damit Kunden tatsächlich mit echtem Geld bezahlen können, müsste zukünftig ein Zahlungsdienstleister wie beispielsweise PayPal oder Stripe eingebunden werden. Dabei müssten jedoch auch Sicherheitsaspekte, Datenschutz, Fehlerbehandlung und eine zuverlässige Zahlungsbestätigung berücksichtigt werden.

8. Erklärung der selbständigen Anfertigung

Hiermit versichern wir

- Leen Anis,
- Esma Cil,
- Patrick Wölflé,
- Dominik Vogler,
- Manuel Schneid,

dass wir die vorliegende Arbeit selbstständig angefertigt und keine anderen als die angegebenen Hilfsmittel verwendet haben. Alle wörtlich oder sinngemäß übernommenen Inhalte wurden entsprechend gekennzeichnet. Für die Erstellung dieser Arbeit wurde IT- / KI-gestützte Schreibwerkzeuge wie z.B. ChatGPT unterstützend verwendet. Die Nutzung umfasst insbesondere die Unterstützung bei der Anwendungsentwicklung, sprachliche Formulierungen und die teilweise Erstellung von Abbildungen.

Kempten, den 09.07.2026



Leen Anis



Esma Cil



Patrick Wölflé



Dominik Vogler



Manuel Schneid

9. Anhang

9.1. RACI

Datum:

17.06.2026

Projekt:

Onlineshop

		Rollen				
		Leen Anis	Esma Cil	Patrick Wölfle	Dominik Vogler	Manuel Schneid
AP.-Nr. 1	Projektplanung					
1.1	Zieldefinition	C	R	C	C	C
1.2	Organisation	C	C	C	C	R
AP.-Nr. 2	Analyse					
2.1	Anforderungen	R	C	C	C	C
2.2	Recherche	C	C	R	C	C
AP.-Nr. 3	Design und Layout					
AP.-Nr. 4	Entwicklung					
4.1	Backend	C	C	C	C	R
4.2	Frontend	C	C	C	R	C
AP.-Nr. 5	Testphase					
5.1	Funktionstest	C	R	C	C	C
5.2	Fehlerbehebung	C	C	C	R	C
AP.-Nr. 6	Abschluss					
6.1	Dokumentation	R	C	C	C	C
6.2	Präsentation	C	C	R	C	C

R	Responsible
A	Accountable
C	Consulted
I	Informed

Durchführungsverantwortung
 Kostenverantwortung
 Muss bei Durchführung eingebunden, gefragt werden
 Ist über Fortschritt zu informieren

9.2. Projektplan

Projektplan

PROJEKTNAME	Blubber Baron Onlineshop
TEAMNAME	wird noch nachgereicht
Startdatum	Montag, 13. April 2026
Enddatum	Donnerstag, 9. Juli 2026

NR.	ARBEITSPAKET	Start	Ende	Tage	APRIL			MAI				JUNI				JULI	
					KW16	KW17	KW18	KW19	KW20	KW21	KW22	KW23	KW24	KW25	KW26	KW27	KW28
1.	Projektplanung	13.04.26	21.04.26	9													
1.1.	Zieldefinition	13.04.26	17.04.26	5													
1.2.	Organisation	18.04.26	21.04.26	4													
2.	Analyse	22.04.26	30.04.26	9													
2.1.	Anforderungen	22.04.26	27.04.26	6													
2.2.	Recherche	28.04.26	30.04.26	3													
3.	Design und Layout	01.05.26	08.05.26	8													
4.	Entwicklung	09.05.26	18.06.26	41													
4.1.	Backend	09.05.26	18.06.26	41													
4.2.	Frontend	15.05.26	18.06.26	35													
5.	Testphase	19.06.26	30.06.26	12													
5.1.	Funktionstest	19.06.26	30.06.26	12													
5.2.	Fehlerbehebung	19.06.26	30.06.26	12													
6.	Abschluss	01.07.26	09.07.26	9													
6.1.	Dokumentation	03.06.26	08.07.26	36													
6.2.	Präsentation	06.07.26	09.07.26	4													

9.3. Strukturplan

